GloveUp — Documentación para Examen

Alumno: Santiago Yu

Proyecto: GloveUp — Sistema de Gestión para Boxeo

Tecnologías: Node.js, Express, MongoDB, React, HTML/CSS/JS

Fecha: Marzo 2026

📘 ASIGNATURA 1 — Sistemas de Gestión Empresarial (SGE)

1. Tipos de sistemas de gestión empresarial

Introducción: Un sistema de gestión empresarial (SGE o SSGG) es un software que automatiza y optimiza los procesos internos de una organización. Existen múltiples tipos según el área de negocio que abordan, desde la gestión integral de recursos hasta la relación con clientes o la logística.

Los principales tipos de SSGG son:

Sigla	Nombre completo	Propósito
ERP	Enterprise Resource Planning	Gestión integral: finanzas, inventario, RRHH, producción
CRM	Customer Relationship Management	Gestión de relaciones con clientes, ventas, marketing
SCM	Supply Chain Management	Gestión de la cadena de suministro
HRM	Human Resource Management System	Gestión de recursos humanos, nóminas, turnos
CMS	Content Management System	Gestión de contenido web (WordPress, Joomla)
BPM	Business Process Management	Automatización y optimización de procesos
BI	Business Intelligence	Análisis de datos, dashboards, informes
PM	Project Management	Gestión de proyectos y tareas (Jira, Trello)
SGA	Sistema de Gestión de Almacenes	Control de stock e inventario
SGD	Sistema de Gestión Documental	Gestión y archivo de documentos
POS	Point of Sale	Punto de venta, facturación
LMS	Learning Management System	Gestión de formación y aprendizaje

Justificación: Conocer los tipos de SSGG permite ubicar cualquier software empresarial dentro del ecosistema tecnológico. GloveUp, al combinar funcionalidades de varios tipos, demuestra que un proyecto real rara vez encaja en una sola categoría.

2. Clasificación del software desarrollado

> **Introducción:** GloveUp es una plataforma web para la gestión integral de gimnasios de boxeo y deportes de contacto. Conecta entrenadores con boxeadores, gestiona alumnos, facilita sesiones de sparring y proporciona métricas de negocio en tiempo real.

GloveUp es un **Sistema de Gestión Empresarial Vertical/Sectorial** especializado en el sector del boxeo. Se clasifica como:

Tipo	Justificación
CRM deportivo	Gestiona la relación entre entrenadores y boxeadores (clientes). El entrenador administra altas, bajas y seguimiento de sus alumnos.
ERP vertical	Integra múltiples funciones de negocio de un gimnasio de boxeo: gestión de alumnos, facturación (precio mensual × inscripciones), agenda/calendario, sparring.
BPM parcial	Automatiza procesos como el flujo de retos de sparring: enviar → aceptar/rechazar → sesión → completar → reseña.
BI parcial	Genera dashboards con métricas (boxeadores activos, inscripciones, ingresos estimados) y gráficos doughnut/línea.

Características clave del software:

- **Multi-rol**: Boxeador y Entrenador con interfaces y funcionalidades diferenciadas
- **Gestión de alumnos**: CRUD completo de boxeadores asignados a un entrenador
- **Gestión económica**: Precio mensual configurable, cálculo automático de ingresos
- **Sistema de sparring**: Búsqueda de compañeros, retos, sesiones y reseñas
- **Gestión de gimnasios**: Registro, ficha con fotos, bio y geolocalización
- **Dashboard con KPIs**: Métricas visuales en tiempo real
- **Calendario integrado**: Vista mensual con eventos de sparring, inscripciones y recordatorios
- **Seguridad**: Hashing de contraseñas con bcrypt y cifrado AES-256-CBC de datos sensibles

> **Justificación:** GloveUp no es un simple CRUD. Al integrar funciones CRM, ERP, BPM y BI en un solo producto vertical, demuestra cómo los sistemas de gestión modernos combinan múltiples paradigmas para cubrir las necesidades reales de un sector específico.

---

3. Gestor de bases de datos utilizado

> **Introducción:** La elección del gestor de base de datos es una de las decisiones arquitectónicas más importantes en cualquier proyecto. Afecta al rendimiento, la flexibilidad del modelo de datos y la velocidad de desarrollo.

Propiedad	Valor

SGBD	**MongoDB** (versión Community)
Tipo	NoSQL — Orientado a documentos
ODM	Mongoose v7.6.0 (Object Document Mapper para Node.js)
URI de conexión	`mongodb://127.0.0.1:27017/gloveup`
Puerto	27017 (por defecto de MongoDB)

¿Por qué MongoDB?

1. **Flexibilidad de esquema**: Los documentos de boxeadores tienen subdocumentos anidados complejos (historial de sparring, retos enviados/recibidos, sesiones con reseñas). En SQL esto requeriría 6-8 tablas con JOINs; en MongoDB es un solo documento.
2. **Datos semi-estructurados**: Campos como `fotos` (array de strings), `sparringHistory` (array de objetos) se almacenan naturalmente como subdocumentos JSON.
3. **Velocidad de desarrollo**: Con Mongoose definimos schemas directamente en JavaScript, sin necesidad de migraciones SQL ni ORM pesado.
4. **Escalabilidad horizontal**: MongoDB soporta sharding nativo, útil si la aplicación crece.
5. **Ecosistema Node.js**: Integración nativa con Express.js y el stack MERN/MEAN.

> **Justificación:** MongoDB se eligió porque el modelo de datos de GloveUp es inherentemente jerárquico (un boxeador contiene retos, sesiones, reseñas anidadas). Una base de datos relacional habría requerido múltiples tablas y JOINs complejos, mientras que MongoDB almacena toda esa información como un documento coherente.

---

4. Operaciones de documentación de la aplicación

> **Introducción:** La documentación es esencial para la mantenibilidad de cualquier software. GloveUp implementa múltiples estrategias de documentación, desde generación automática hasta documentación inline en el código.

Operación	Descripción	Archivo
Generador de reportes	Script Python (`doc_generator.py`) que escanea todo el proyecto y genera un informe Markdown con árbol de directorios + código fuente	`tools/doc_generator.py`
Resúmenes con IA	El generador puede activar Ollama (IA local) para resumir automáticamente cada archivo y carpeta	Flag `-ia` del script
Documentación de API	Los endpoints están organizados en archivos de rutas con nombres descriptivos (`auth.js` , `boxeadores.js` , `entrenadores.js` , `gimnasios.js`)	`server/src/routes/`

Esquemas de datos	Los modelos Mongoose documentan la estructura de cada colección: tipos, validaciones, valores por defecto	`server/src/models/`
Variables de entorno	Archivo `.env` documenta los parámetros de configuración del servidor	`server/.env`
Comentarios en código	Comentarios en español e inglés en archivos CSS, JS y HTML explicando secciones y decisiones	Todo el proyecto

> **Justificación:** La documentación automática con IA local (Ollama) es un enfoque innovador que mantiene la documentación sincronizada con el código sin esfuerzo manual. Esto reduce el problema habitual de la documentación desactualizada.

---

5. Módulos del SSGG

> **Introducción:** La arquitectura modular permite que cada parte del sistema sea independiente, facilitando el desarrollo, el testing y el mantenimiento. GloveUp está organizado en módulos funcionales, cada uno con sus propios archivos HTML, CSS y JS.

```
GLOVEUP/
├── auth/           → Módulo de Autenticación
├── home/           → Módulo Core (estilos globales + lógica compartida)
├── dashboard/
│   ├── boxeador/   → Módulo Dashboard Boxeador
│   └── entrenador/ → Módulo Dashboard Entrenador
├── sparring/       → Módulo de Sparring
├── gyms/           → Módulo de Gimnasios
├── profile/        → Módulo de Perfil
├── admin/          → Módulo de Administración
├── assets/         → Recursos estáticos (imágenes, vídeos)
└── server/         → Módulo Backend (API + BBDD)
    └── src/
        ├── models/     → Modelos de datos
        ├── routes/     → Lógica de negocio + API
        ├── utils/      → Utilidades (cifrado, seguridad)
        └── seed/       → Datos iniciales
```

Módulo	Propósito	Archivos clave
Auth	Login, Registro, Recuperación de contraseña	`auth/index.html`, `auth/auth.js`, `auth/auth.css`

Home/Core	Diseño global, sidebar, tema claro/oscuro, lógica de roles	`home/styles.css` , `home/home.js`
Dashboard Boxeador	Panel con KPIs: sparrings/mes, minutos, pendientes	`dashboard/boxeador/dashboard.react.js`
Dashboard Entrenador	Estadísticas, CRUD boxeadores, calendario, gimnasio, precios	`dashboard/entrenador/dashboard.react.js`
Sparring	Búsqueda de sparring partners, retos, sesiones, reseñas	`sparring/index.html`
Gyms	Exploración de gimnasios, ficha detallada, fotos, mapa	`gyms/index.html` , `gyms/gyms.js`
Profile	Ficha personal, historial de sparrings, edición de datos	`profile/index.html` , `profile/profile.js`
Admin	Tabla de usuarios registrados (herramienta interna)	`admin/users_list.html`
Server	API REST, conexión a MongoDB, lógica de negocio, seguridad	`server/src/index.js` , `routes/` , `models/` , `utils/`

Justificación: Cada módulo es autocontenido (HTML + CSS + JS propios). Esto permite desarrollo paralelo, mantenimiento aislado (un cambio en Sparring no afecta a Profile), y carga lazy de CSS/JS (solo se carga lo necesario).

---

6. Parámetros configurables

Introducción: Un sistema configurable permite adaptar el comportamiento del software sin modificar el código fuente. GloveUp define parámetros en tres niveles: backend, frontend y diseño visual.

a) Parámetros de Backend (`server/.env`)

Parámetro	Valor actual	Propósito
`PORT`	`3000`	Puerto donde escucha el servidor Express
`MONGO_URI`	`mongodb://127.0.0.1:27017/gloveup`	Cadena de conexión a MongoDB
`ENCRYPTION_KEY`	`ae157034...` (64 hex)	Clave AES-256-CBC para cifrar datos sensibles

b) Parámetros de Frontend (`localStorage`)

Clave	Valores posibles	Propósito

`gloveup_user_email`	Email del usuario	Identifica al usuario logueado
`gloveup_user_role`	`boxeador` / `entrenador`	Control de visibilidad por roles en la UI
`gloveup_user_nombre`	Nombre del usuario	Muestra el nombre en la interfaz
`gloveup_api_base_url`	URL del servidor	Permite cambiar el endpoint de la API
`theme`	`light` / `dark`	Tema visual de la aplicación

c) Parámetros de Diseño CSS (variables `:root`)

Variable	Valor	Propósito
`--color-primary`	`#0a0a0a`	Color principal (negro)
`--color-secondary`	`#ffffff`	Color secundario (blanco)
`--color-accent`	`#111827`	Color de acento (gris oscuro)
`--color-bg-body`	`#f8fafc`	Fondo general de la app
`--font-main`	`'Inter', system-ui...`	Tipografía de cuerpo
`--font-heading`	`'Outfit', 'Inter'...`	Tipografía de títulos
`--radius-xl`	`24px`	Radio de borde de tarjetas
`--transition-base`	`250ms`	Duración de transiciones
`--glass-bg`	`rgba(255,255,255,0.72)`	Efecto glassmorphism

Justificación: Todos los colores, tipografías y espaciados se definen como variables CSS en `:root`. Cambiar una sola variable modifica toda la aplicación consistentemente. El tema oscuro se implementa redefiniendo estas variables en `body.theme-dark`.

---

7. Comunicación con otros sistemas

Introducción: Ningún sistema empresarial funciona de forma aislada. GloveUp se comunica con servicios externos para cargar recursos (tipografías, librerías) y mantiene comunicación interna entre sus capas frontend, backend y base de datos.

Dirección	Sistema externo	Tipo	Descripción
Entrada	Google Fonts CDN	HTTP GET	Tipografías Inter y Outfit
Entrada	Chart.js CDN	HTTP GET	Librería de gráficos

Entrada	FullCalendar CDN	HTTP GET	Librería de calendario
Entrada	Font Awesome CDN	HTTP GET	Iconos
Entrada/Salida	Ollama (IA local)	HTTP POST/JSON	Resúmenes generados por IA
Interna	MongoDB ↔ Express	TCP/27017	Comunicación mediante Mongoose
Interna	Frontend ↔ Backend	HTTP REST/JSON	API REST con `fetch()`

Justificación: La comunicación Frontend → Backend se realiza mediante peticiones `fetch()` con formato JSON. No hay integración con APIs externas de terceros (pasarelas de pago, etc.), pero la arquitectura REST lo permite fácilmente.

---

8. Entornos de desarrollo y tecnologías

Introducción: El stack tecnológico define las herramientas utilizadas en cada capa del sistema. GloveUp emplea tecnologías modernas del ecosistema JavaScript, complementadas con Python para automatización.

Stack tecnológico

Capa	Tecnología	Versión	Propósito
Frontend	HTML5	—	Estructura semántica
Frontend	CSS3 (Vanilla)	—	Estilos, responsive, tema claro/oscuro
Frontend	JavaScript ES6+	—	Lógica de UI, fetch API, DOM
Frontend	React 18	CDN	Componentes del dashboard
Backend	Node.js	v24.x	Runtime del servidor
Backend	Express.js	v4.18.2	Framework HTTP / API REST
Backend	Mongoose	v7.6.0	ODM para MongoDB
Backend	bcrypt	latest	Hashing de contraseñas
BBDD	MongoDB	Community	Base de datos NoSQL documental
Herramientas	Python 3	v3.x	Script de documentación
Librerías	Chart.js	CDN	Gráficos doughnut y línea
Librerías	FullCalendar	CDN	Calendario interactivo

Entorno de desarrollo

Herramienta	Uso
VS Code	Editor de código principal
PowerShell	Ejecución de servidor y scripts
Git	Control de versiones
npm	Gestor de paquetes
nodemon	Recarga automática del servidor

> **Justificación:** Se eligió el stack MERN (MongoDB, Express, React, Node) por su coherencia (JavaScript en todas las capas), su gran comunidad y la facilidad para construir APIs REST con Express y consumirlas desde el frontend.

---

9. Tablas (colecciones) y campos de la base de datos

> **Introducción:** En MongoDB no hay "tablas" sino **colecciones** con **documentos** (JSON). Equivalencia: Colección = Tabla, Documento = Fila, Campo = Columna. GloveUp define 4 colecciones principales.

Colección: `usuarios`

Campo	Tipo	Restricciones	Descripción
`_id`	ObjectId	PK, auto	Identificador único
`nombre`	String	required, trim	Nombre completo
`email`	String	required, unique, lowercase	Email (login)
`password`	String	required	Contraseña (hasheada con bcrypt)
`dniLicencia`	String	unique, sparse	DNI o licencia (cifrado AES-256)
`rol`	String	enum: `boxeador` , `entrenador` , `admin`	Rol del usuario
`createdAt`	Date	auto	Fecha de creación

Colección: `boxeadores`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único
`usuarioId`	ObjectId	FK → usuarios

`entrenadorId`	ObjectId	FK → entrenadores
`nombre`	String	Nombre del boxeador
`email`	String	Email (identificador)
`nivel`	String	Principiante, Intermedio, Avanzado, Amateur, Profesional
`disciplina`	String	Deporte practicado
`peso` , `altura` , `alcance`	String	Datos físicos
`sparringHistory`	Array	Historial: `{date, partner, place, notes}`
`sparringChallengesSent`	Array	Retos enviados
`sparringChallengesReceived`	Array	Retos recibidos
`sparringSessions`	Array	Sesiones con reseñas
`fechaInscripcion`	Date	Fecha de inscripción al gimnasio

Colección: `entrenadores`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único
`usuarioId`	ObjectId	FK → usuarios
`nombre` , `email`	String	Datos personales
`especialidad`	String	Especialidad de entrenamiento
`gimnasio`	String	Nombre del gimnasio
`precioMensual`	Number	Precio mensual del servicio

Colección: `gimnasios`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único
`nombre` , `key`	String	Nombre y clave normalizada
`ubicacion` , `direccion`	String	Localización
`lat` , `lng`	Number	Geolocalización
`bio`	String	Descripción

`fotos`	Array	URLs o base64 de fotos
`creadoPorEmail`	String	Entrenador que lo creó

Justificación: El modelo de datos está diseñado para aprovechar la naturaleza documental de MongoDB. Los datos de sparring (retos, sesiones, reseñas) se almacenan como subdocumentos anidados dentro del boxeador, evitando JOINs innecesarios y mejorando el rendimiento de lectura.

---

10. Consultas a la base de datos

Introducción: Las consultas son el mecanismo mediante el cual el backend lee y escribe datos en MongoDB. GloveUp implementa unas 35 consultas distintas organizadas por módulo.

Módulo Auth (`/api/auth`) — 8 consultas

Ruta	Método	Consulta	Descripción
`/register`	POST	`Usuario.findOne({email})`	Verificar email existente
`/register`	POST	`Usuario.create({...})`	Crear usuario (password hasheada)
`/login`	POST	`Usuario.findOne({$or: [{email}, {nombre}]})`	Buscar usuario
`/forgot-password`	POST	`user.save()`	Guardar nueva contraseña hasheada

Módulo Boxeadores (`/api/boxeadores`) — 10 consultas

Ruta	Método	Descripción
`/`	GET	Listar todos los boxeadores
`/lookup`	GET	Buscar boxeador por email
`/challenges`	POST	Enviar reto de sparring
`/sessions/complete`	POST	Completar una sesión
`/me`	GET/PUT	Obtener/actualizar perfil propio

Módulo Entrenadores (`/api/entrenadores`) — 12 consultas

Ruta	Método	Descripción

`/me/metricas`	GET	Calcular KPIs del entrenador
`/me/boxeadores`	GET/POST/DELETE	CRUD de boxeadores asignados
`/me/cobros`	GET	Calcular cobros e ingresos

Módulo Gimnasios (`/api/gimnasios`) — 4 consultas

Ruta	Método	Descripción
`/`	GET/POST	Listar / Crear-actualizar gimnasio
`/lookup`	GET	Buscar gimnasio por nombre

Total: ~35 consultas distintas a la base de datos.

> **Justificación:** Las consultas están organizadas por recurso REST, siguiendo las convenciones de diseño de APIs RESTful. Cada módulo gestiona su propia responsabilidad de datos.

---

11. Interfaces de acceso a datos

> **Introducción:** La aplicación implementa una arquitectura de 3 capas que separa la presentación, la lógica de negocio y el acceso a datos, siguiendo el patrón MVC.

```
┌──────────────┐        fetch()        ┌──────────────┐        Mongoose        ┌──────────┐
│   FRONTEND   │── HTTP/JSON ──→       │   BACKEND    │── ODM ──────────→      │ MongoDB  │
│  (Browser)   │←─ JSON ──────         │  (Express)   │←─ Documents ──         │ (27017)  │
└──────────────┘                       └──────────────┘                        └──────────┘
```

Capa 1 — Frontend: El JavaScript del navegador usa `fetch()` para peticiones HTTP a `http://localhost:3000/api/*`. Los datos se reciben como JSON y se renderizan en el DOM.

Capa 2 — Backend: Express.js recibe las peticiones y las enruta al controlador adecuado. Cada ruta valida datos, ejecuta la lógica de negocio y devuelve JSON.

Capa 3 — Base de datos: Mongoose define los schemas que validan los datos antes de guardarlos. La conexión es persistente durante todo el ciclo de vida del servidor.

> **Justificación:** Esta separación en capas permite que cada nivel evolucione independientemente. Se podría reemplazar el frontend por una app móvil sin tocar el backend, o migrar de MongoDB a otra BBDD sin modificar el frontend.

---

12. Informes y gráficas

Introducción: Las gráficas y los informes transforman datos brutos en información visual comprensible. GloveUp utiliza Chart.js para generar gráficas interactivas y FullCalendar para la vista temporal de eventos.

Dashboard Boxeador (3 gráficas doughnut)

Gráfica	Datos	Color
Sparrings este mes	Nº sparrings vs meta 10	`#111827`
Minutos de sparring	Minutos vs meta 300	`#6b7280`
Sparrings pendientes	Pendientes vs 10	`#9ca3af`

Dashboard Entrenador (4 doughnut + 1 línea)

Gráfica	Tipo	Datos
Tu gimnasio	Doughnut	Indicador de gimnasio configurado
Boxeadores activos	Doughnut	Nº alumnos vs capacidad
Inscripciones este mes	Doughnut	Altas del mes actual
Ingresos estimados	Doughnut	precio × inscripciones
Ingresos acumulados	Línea	Evolución temporal de ingresos

Calendario interactivo (Entrenador)

Tipo de evento	Color	Origen
Sparring	Negro	`sparringHistory`
Inscripción	Gris	`fechaInscripcion`
Recordatorio	Gris claro	Generado automáticamente

Justificación: Los gráficos doughnut fueron elegidos porque comunican de forma inmediata el progreso hacia un objetivo (porcentaje de completitud). La paleta monocromática (negro/gris) mantiene coherencia con el diseño premium de la aplicación.

💻 ASIGNATURA 2 — Programación de Servicios y Procesos (PSP)

1. Comunicación entre sistemas — API REST

> **Introducción:** Una API REST (Representational State Transfer) es un estilo arquitectónico que permite la comunicación entre sistemas mediante el protocolo HTTP. Cada recurso se identifica con una URL y se manipula con los métodos HTTP estándar (GET, POST, PUT, DELETE).

GloveUp implementa una **API REST** como mecanismo de comunicación entre el frontend y el backend:

Propiedad	Valor
Protocolo	HTTP/1.1
Formato	JSON (peticiones y respuestas)
Base URL	`http://localhost:3000/api`
Autenticación	Sesión basada en email (localStorage)

Endpoints organizados por recurso:

Recurso	Endpoints	Operaciones
`/api/auth`	3	Login, Registro, Recuperar contraseña
`/api/boxeadores`	8	CRUD, Retos, Sesiones, Perfil
`/api/entrenadores`	11	CRUD, Métricas, Gestión boxeadores, Cobros
`/api/gimnasios`	4	CRUD, Búsqueda, Detalle
`/api/usuarios`	1	Listado
Total	**27 endpoints**	

Ejemplo de flujo de comunicación:

```
CLIENTE                                    SERVIDOR
  │                                            │
  │── POST /api/auth/login ──────────────────→│
  │   Body: {"email":"user@test.com",          │
  │          "password":"123456"}              │
```

```
│                                          │ ← bcrypt.compare()
│←─ 200 OK ──────────────────────────────│
│   Body: {"id":"...", "nombre":"Juan",    │
│          "email":"...", "rol":"boxeador"} │
```

Justificación: REST fue elegido por su simplicidad, su compatibilidad universal con cualquier cliente HTTP, y porque los recursos de GloveUp (boxeadores, entrenadores, gimnasios) se mapean naturalmente a URLs RESTful.

---

2. Rol de servidor y rol de cliente

Introducción: En una arquitectura cliente-servidor, cada parte tiene responsabilidades claramente definidas. El servidor procesa y almacena datos; el cliente los presenta al usuario y gestiona la interacción.

Servidor (el que SIRVE la API)

```javascript
// server/src/index.js
import express from 'express';
const app = express();

app.use(cors());                          // Permite peticiones cross-origin
app.use(express.json({ limit: '50mb' })); // Parsea el body como JSON

app.use('/api/boxeadores', boxeadoresRouter);
app.use('/api/entrenadores', entrenadoresRouter);
app.use('/api/gimnasios', gimnasiosRouter);
app.use('/api/auth', authRouter);

app.listen(3000);
```

Responsabilidades: Recibir peticiones HTTP → Validar datos → Ejecutar lógica de negocio → Acceder a la BBDD → Devolver JSON con códigos HTTP (200, 201, 400, 401, 404, 500).

Cliente (el que CONSUME la API)

```javascript
// Ejemplo de dashboard/boxeador/dashboard.react.js
const requestJson = (path) => {
    return fetch(`${API_BASE_URL}${path}`, {
```

```
        method: 'GET',
        headers: { 'Content-Type': 'application/json' }
    }).then(async (res) => {
        const payload = await res.json().catch(() => ({}));
        if (!res.ok) throw new Error(payload.error || `Error ${res.status}`);
        return payload;
    });
};
```

Responsabilidades: Construir peticiones HTTP → Enviar al servidor → Procesar JSON → Renderizar datos en la UI → Gestionar errores.

> **Justificación:** La separación cliente-servidor permite que el frontend sea una SPA (Single Page Application) que solo intercambia datos JSON con el backend, reduciendo el ancho de banda y mejorando la experiencia de usuario respecto a una aplicación web tradicional con recarga de página completa.

---

3. Apertura de servicio web — Documentación del contrato

> **Introducción:** Documentar el contrato de una API permite que desarrolladores externos (o futuros mantenedores) puedan consumirla sin necesidad de leer el código fuente. Se definen las rutas, los métodos, los formatos de entrada/salida y los posibles códigos de error.

🔵 **POST** `/api/auth/login` — Iniciar sesión

Request:

```
{
  "email": "juan@email.com",
  "password": "miContrasena123"
}
```

Response 200 OK:

```
{
  "id": "665f1a2b3c4d5e6f7a8b9c0d",
  "nombre": "Juan Pérez",
  "email": "juan@email.com",
  "rol": "boxeador",
```

```
  "dniLicencia": "12345678A"
}
```

Response 401 Unauthorized:

```
{ "error": "Credenciales inválidas" }
```

🟢 **GET** `/api/boxeadores/me?email={email}` — Perfil de boxeador

Response 200 OK:

```
{
  "_id": "665f1a2b...",
  "nombre": "Juan Pérez",
  "nivel": "Amateur",
  "sparringHistory": [
    { "date": "2026-03-15", "partner": "Carlos", "place": "GymTest" }
  ],
  "sparringSessions": [...]
}
```

🟡 **POST** `/api/gimnasios` — Crear/actualizar gimnasio

Request:

```
{
  "nombre": "Boxing Club Madrid",
  "creadoPorEmail": "coach@email.com",
  "bio": "Gimnasio especializado en boxeo olímpico",
  "fotos": ["..."]
}
```

> **Justificación:** Esta documentación sigue el estándar de la industria para APIs REST. Al incluir ejemplos reales de request/response con códigos HTTP, cualquier desarrollador puede integrar GloveUp con sistemas externos sin necesidad de soporte adicional.

---

4. Técnicas criptográficas de ida y vuelta (con desencriptación)

> **Introducción:** El cifrado simétrico permite proteger datos sensibles que necesitan ser recuperados en su forma original. A diferencia del hashing (unidireccional), el cifrado es reversible: se puede cifrar (ida) y descifrar (vuelta) utilizando una clave secreta. GloveUp implementa AES-256-CBC para proteger datos como el DNI/Licencia.

Implementación real en GloveUp (`server/src/utils/crypto.js`):

```
import crypto from 'crypto';

const ALGORITHM = 'aes-256-cbc';
const IV_LENGTH = 16;
const SECRET_KEY = process.env.ENCRYPTION_KEY; // 32 bytes (64 hex)

// CIFRAR (ida)
export function encrypt(text) {
    if (!text || !isEncryptionEnabled()) return text;
    const iv = crypto.randomBytes(IV_LENGTH);
    const cipher = crypto.createCipheriv(ALGORITHM, Buffer.from(SECRET_KEY, 'hex'),
iv);
    let encrypted = cipher.update(text, 'utf8', 'hex');
    encrypted += cipher.final('hex');
    return iv.toString('hex') + ':' + encrypted;  // iv:cifrado
}

// DESCIFRAR (vuelta)
export function decrypt(encryptedText) {
    if (!encryptedText || !encryptedText.includes(':')) return encryptedText;
    const [ivHex, encrypted] = encryptedText.split(':');
    const iv = Buffer.from(ivHex, 'hex');
    const decipher = crypto.createDecipheriv(ALGORITHM, Buffer.from(SECRET_KEY,
'hex'), iv);
    let decrypted = decipher.update(encrypted, 'hex', 'utf8');
    decrypted += decipher.final('utf8');
    return decrypted;
}
```

¿Dónde se usa en GloveUp?

Operación	Función	Contexto

Registro	`encrypt(cleanDni)`	El DNI se cifra antes de guardarse en la BBDD
Login	`decrypt(user.dniLicencia)`	El DNI se descifra para devolverlo al usuario
Forgot-password	`decrypt(user.dniLicencia)`	Se descifra el DNI para verificar la identidad

Resultado en MongoDB:

```
// Antes (inseguro):
dniLicencia: "12345678A"

// Después (cifrado AES-256-CBC):
dniLicencia: "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6:d4e5f6a7b8c9d0e1f2..."
```

Justificación: Se usa cifrado AES-256-CBC (y no hashing) porque el DNI es un dato que necesitamos recuperar en su forma original (a diferencia de la contraseña). AES-256 es el estándar de cifrado simétrico más seguro y ampliamente utilizado, recomendado por el NIST.

---

5. Técnicas de hashing

Introducción: El hashing es un proceso criptográfico **unidireccional**: transforma datos en un resumen de longitud fija que no se puede revertir. Es la técnica adecuada para almacenar contraseñas, ya que no necesitamos recuperar la contraseña original, solo verificar que coincide.

5.1. Hashing SHA-1 en el sistema de documentación

Implementación real en `tools/doc_generator.py` :

```python
import hashlib

def _sha1(text: str) -> str:
    return hashlib.sha1(text.encode("utf-8", errors="ignore")).hexdigest()
```

Uso: Se calcula el hash SHA-1 del contenido de cada archivo para generar claves de caché. Si el hash no ha cambiado, se reutiliza el resumen IA anterior sin volver a llamar a Ollama.

```python
key = "file:" + _sha1(relpath + "\n" + contenido[:12000])
if key in cache:
    resumen = cache[key]      # Reutilizar (ahorro de tiempo)
```

```
else:
    resumen = resumir_con_ia(...)  # Generar nuevo
    cache[key] = resumen           # Guardar en caché
```

5.2. Hashing bcrypt para contraseñas

Implementación real en `server/src/models/Usuario.js` :

```
import bcrypt from 'bcrypt';
const SALT_ROUNDS = 10;

// Pre-save hook: hashea la contraseña antes de guardar
UsuarioSchema.pre('save', async function (next) {
    if (!this.isModified('password')) return next();
    const salt = await bcrypt.genSalt(SALT_ROUNDS);
    this.password = await bcrypt.hash(this.password, salt);
    next();
});

// Método para comparar contraseñas en el login
UsuarioSchema.methods.comparePassword = async function (candidatePassword) {
    if (this.password.startsWith('$2')) {
        return bcrypt.compare(candidatePassword, this.password);
    }
    return this.password === candidatePassword; // Fallback texto plano
};
```

Resultado en MongoDB:

```
// Antes (inseguro):
password: "miContrasena123"

// Después (hasheado con bcrypt):
password: "$2b$10$djM8K3x9Q2Lp5vN7rY1wOeF4gH6jK..."
```

Propiedades del hashing:

Propiedad	SHA-1	bcrypt
Unidireccional	✅	✅

Determinista	✅	❌ (salt aleatorio)
Longitud fija	40 chars (160 bits)	60 chars
Salt incluido	❌	✅ (protege contra rainbow tables)
Costoso computacionalmente	❌	✅ (protege contra fuerza bruta)
Uso recomendado	Caché, checksums	Contraseñas

> **Justificación:** Se eligió **bcrypt** sobre SHA-1/SHA-256 para contraseñas porque: (1) incluye salt automático, evitando ataques de rainbow tables; (2) es computacionalmente costoso con 10 rondas de sal, haciendo inviable la fuerza bruta; (3) es el estándar de la industria para almacenamiento de contraseñas. SHA-1 se usa para la caché de documentación donde la seguridad no es un requisito, solo la eficiencia.